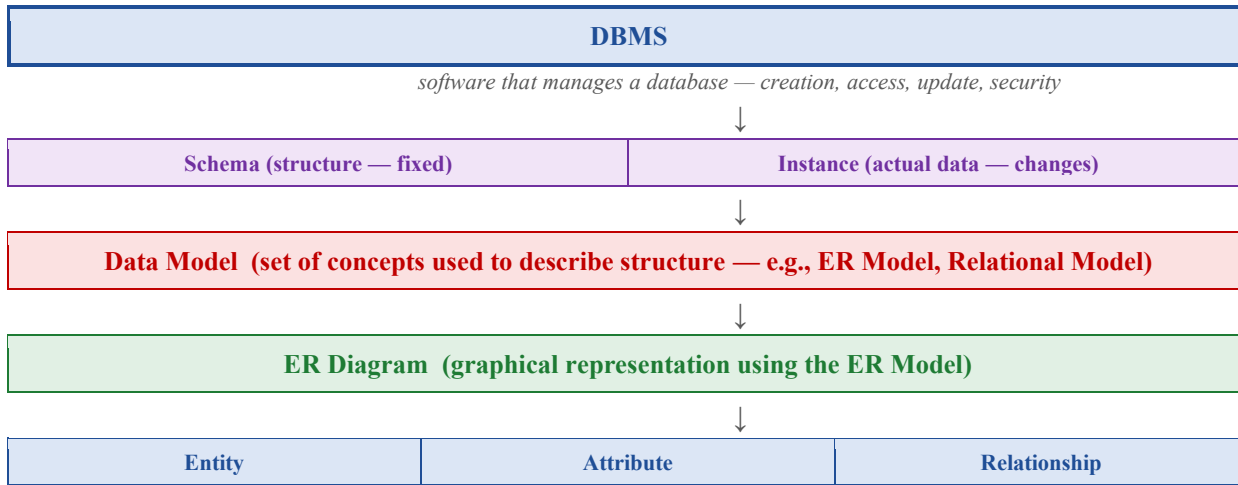


DBMS Concept Map Answer Key

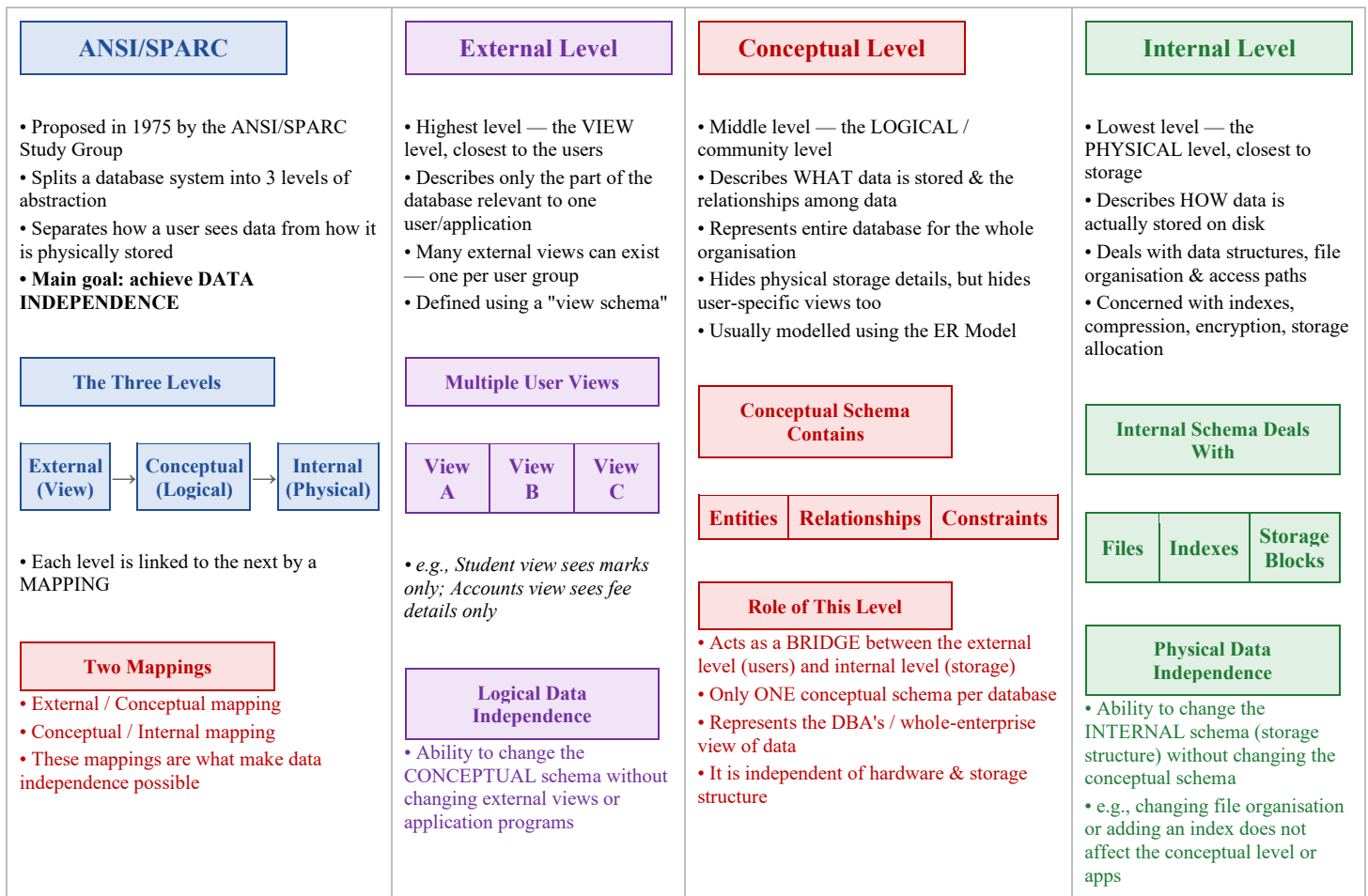
1) DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship



- Entity — real-world object/thing (e.g., Student, Course)
- Attribute — property describing an entity (e.g., Name, RollNo)
- Relationship — association between two or more entities (e.g., Enrols)

- Schema = database "blueprint" — defined once, rarely changes
- Instance = database "snapshot" at a moment — changes with every insert/update/delete

2) ANSI/SPARC Three-Level Architecture — Data Independence at Each Level



Purpose of the Architecture - Hides physical storage complexity from users - Supports many different user views of the same data - Lets structure change without breaking applications	- e.g., adding a new table/attribute to the DB does not disturb existing views Conceptual → External (shielded by mapping)		Internal → Conceptual (shielded by mapping)
---	--	--	--

3) File-Based Systems vs Database Systems — Problems Solved by DBMS

File-Based System — Problem	DBMS — How It Is Solved
Data redundancy — same data duplicated across files	→ Centralized data storage removes duplication
Data inconsistency — duplicate copies go out of sync	→ Single source of truth keeps data consistent
Difficulty in accessing data — needs a new program per query	→ Query language (SQL) allows ad-hoc queries
Data isolation — data scattered across file formats	→ Unified schema integrates all data
Integrity problems — constraints scattered in program code	→ Constraints enforced centrally at DB level
Atomicity problems — partial updates on failure	→ Transaction management ensures all-or-nothing updates
Concurrent access anomalies	→ Concurrency control (locking) manages simultaneous access
Security problems — hard to restrict access per file	→ Authorization & access control at user/role level

4) ER Diagram Components

ER Diagram Components			
↓			
Entity Types	Attribute Types	Relationship Types	Participation Constraints
Entity Types			
Strong Entity		Weak Entity	
- Strong Entity — has its own key attribute, exists independently - Weak Entity — has no key of its own; identified via partial key + identifying relationship to an owner entity; must have TOTAL participation			
Attribute Types			
Simple / Composite	Single- / Multi-valued	Stored / Derived	Key Attribute
- Simple: atomic (Age) Composite: divisible (Name → First+Last) - Single-valued: one value (DOB) Multi-valued: many (PhoneNumbers)			

- Stored: kept in DB (DOB) | Derived: computed (Age from DOB)
- Key attribute: uniquely identifies an entity (RollNo)

Relationship Types

Unary

Binary

Ternary

By degree — number of entity types involved

1 : 1

1 : N

M : N

By cardinality ratio

Participation Constraints

Total (double line) — every instance **MUST** participate

Partial (single line) — participation is optional

5) Hierarchical vs Network vs Relational Data Models

Hierarchical

- Structure: Tree (parent–child)
- Only 1:N relationships allowed
- Access via predefined tree paths
- Rigid — a child can have only 1 parent

Example System

e.g., IBM IMS

Network

- Structure: Graph (owner–member sets)
- Supports M:N via sets
- Access via navigational pointers/sets
- More flexible — a member can have multiple owners

Example System

e.g., IDMS (CODASYL)

Relational

- Structure: Set of tables (relations)
- Any cardinality via keys
- Access via declarative queries (SQL)
- Most flexible — based on set theory & predicate logic

Example System

e.g., MySQL, Oracle

6) EER Model — Generalization, Specialization, Aggregation

Specialization (Top-Down)

Generalization (Bottom-Up)

Aggregation

Specialization — Top-Down

- Start from a higher-level (superclass) entity and divide it into sub-classes based on distinguishing features
- *e.g., Employee → {Engineer, Manager, Technician}*

Generalization — Bottom-Up

- Start from lower-level entities that share common attributes and combine them into a higher-level superclass
- *e.g., Car + Truck → Vehicle*

Constraints on Specialization/Generalization

Disjoint — belongs to only **ONE** subclass

Overlapping — can belong to more than one

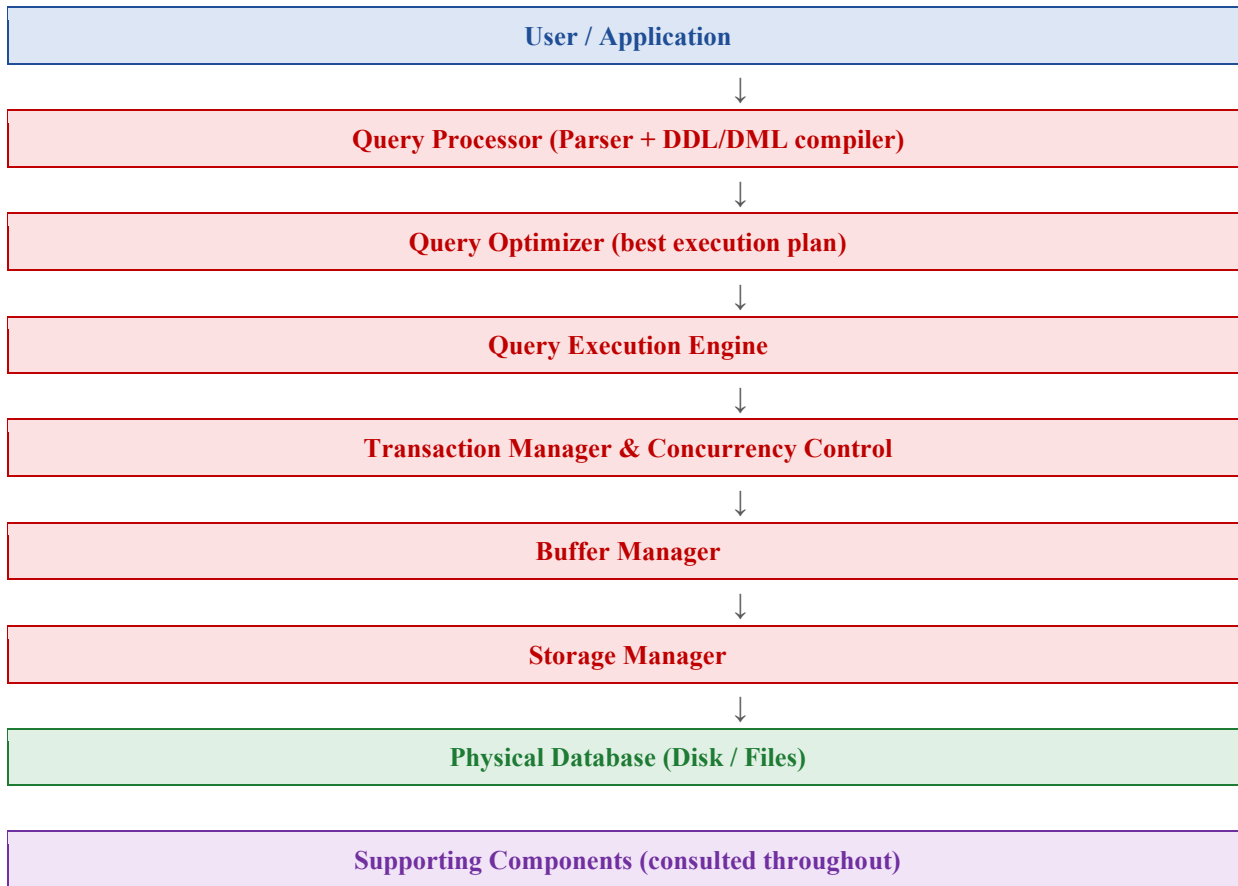
Total — every superclass entity **MUST** specialize

Partial — membership is optional

Aggregation

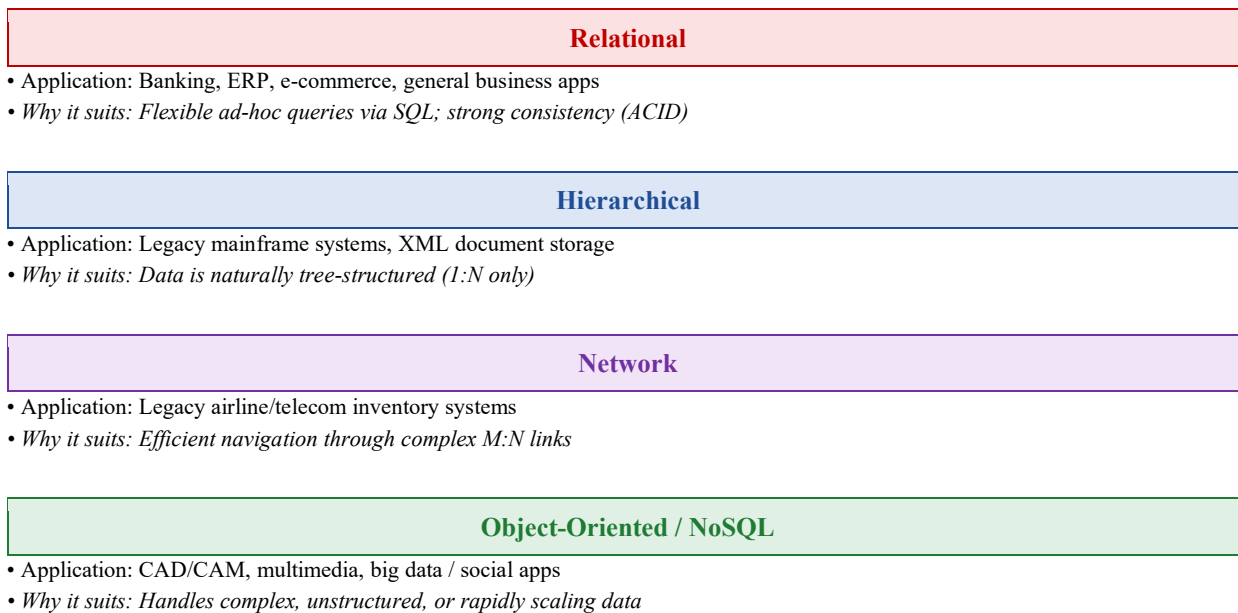
- Treats a relationship (plus its participating entities) as a higher-level "abstract entity" so it can itself participate in another relationship
- Used when a relationship needs to connect to another entity — something a plain ER model cannot express

7) DBMS Software Architecture — Flow of a User Request



- Data Dictionary / Catalog — stores metadata (schema, constraints)
- Recovery Manager — restores consistent state after a crash

8) Data Models Mapped to Real-World Applications



9) Schemas & Instances → Physical / Logical Data Independence

Schema — structure, relatively static	Instance — actual data, changes constantly
--	---

Schema is defined at EACH of the 3 ANSI/SPARC levels (External, Conceptual, Internal)



independence comes from the MAPPINGS between schemas



Logical Data Independence

- Ability to change the Conceptual schema without changing External schemas/apps
- **Mapping involved: Conceptual ↔ External**

Physical Data Independence

- Ability to change the Internal schema without changing the Conceptual schema
- **Mapping involved: Internal ↔ Conceptual**

- Instances change far more often than schemas — the 3-level architecture isolates users from BOTH kinds of change

10) Roles & Responsibilities in a DBMS Environment

DBA	Application Developer	End User
• Defines the conceptual schema & storage structure • Grants authorization / access control • Ensures integrity constraints, backup & recovery • Monitors performance and tunes the database	• Writes application programs using DML (embedded SQL / host language calls) • Builds forms, reports, and interfaces • Works within the DBA-defined schema — does not alter it	• Naive user — fixed forms/menus, no query knowledge • Casual/Sophisticated user — writes ad-hoc SQL queries • Consumes data; not concerned with internal storage